

JavaScript Functions Review

JavaScript Functions

- Functions are reusable blocks of code that perform a specific task.
- Functions can be defined using the `function` keyword followed by a name, a list of parameters, and a block of code that performs the task.

```
1 function addNumbers(x, y, z) {  
2   return x + y + z;  
3 }  
4  
5 console.log(addNumbers(5, 3, 8)); // Output: 16
```

16

- Arguments are values passed to a function when it is called.
- A function call is the process of executing a function in a program by specifying the function's name followed by parentheses, optionally including arguments inside the parentheses.
- When a function finishes its execution, it will always return a value.
- By default, the return value of a function is `undefined`.
- The `return` keyword is used to specify the value to be returned from the function and ends the function execution.
- Default parameters allow functions to have predefined values that will be used if an argument is not provided when the function is called. This makes functions more flexible and prevents errors in cases where certain arguments might be omitted.

```
1 const calculateTotal = (amount, taxRate = 0.05) => {  
2   return amount + (amount * taxRate);  
3 };  
4  
5 console.log(calculateTotal(100)); // Output: 105
```

105

```
5 console.log(multiplyNumbers(4, 5)); // Output: 20
```

Arrow Functions

- Arrow functions are a more concise way to write functions in JavaScript.

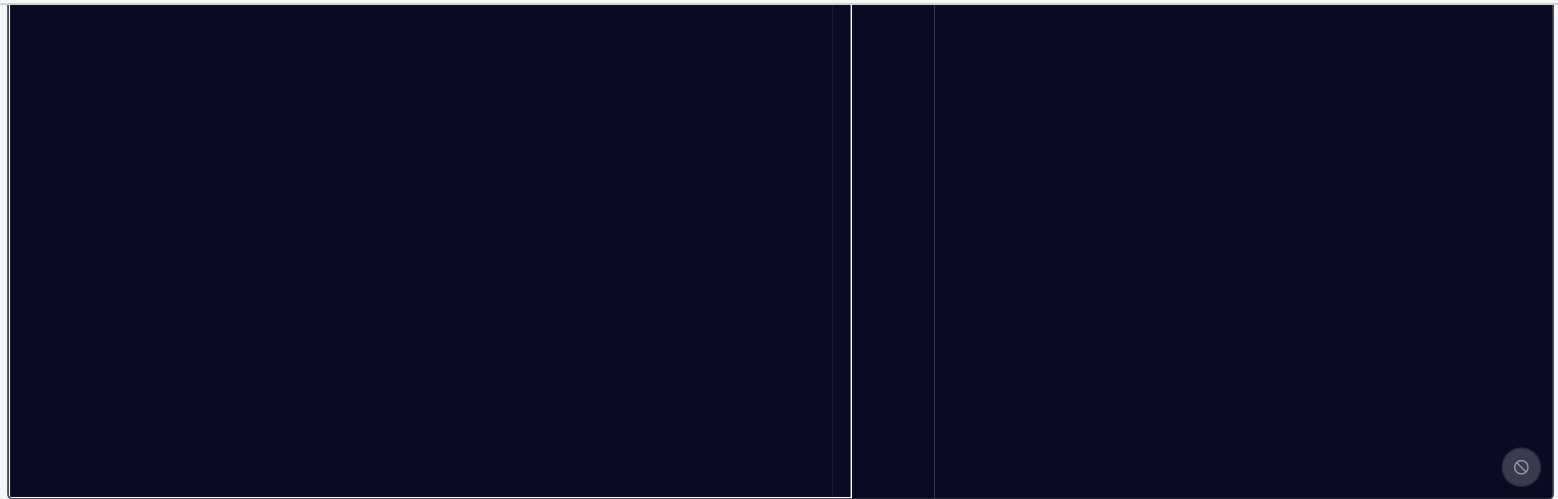
```
1 const calculateArea = (length, width) => {  
2   const area = length * width;  
3   return `The area of the rectangle is ${area} square units.`;  
4 };  
5  
6 console.log(calculateArea(5, 10)); // Output: "The area of the rectangle
```

```
"The area of the rectangle is 50 square units."
```

- When defining an arrow function, you do not need the `function` keyword.
- If you are using a single parameter, you can omit the parentheses around the parameter list.

```
1 const cube = x => {  
2   return x * x * x;  
3 };  
4  
5 console.log(cube(3)); // Output: 27
```

```
27
```



Scope in Programming

- **Global scope:** This is the outermost scope in JavaScript. Variables declared in the global scope are accessible from anywhere in the code and are called global variables.
- **Local scope:** This refers to variables declared within a function. These variables are only accessible within the function where they are declared and are called local variables.
- **Block scope:** A block is a set of statements enclosed in curly braces `{ }` such as in `if` statements, or loops.
- Block scoping with `let` and `const` provides even finer control over variable accessibility, helping to prevent errors and make your code more predictable.

Assignment

- ☐ Review the JavaScript Functions topics and concepts.

Submit

Ask for Help